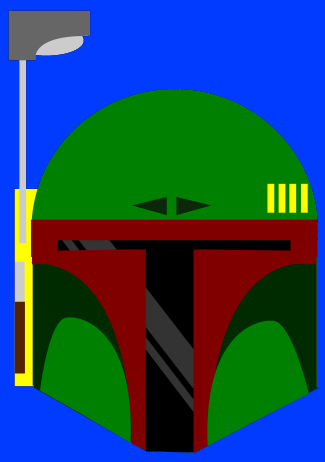
{EPITECH}

RTC STRIKES BACK_

< BOOTSTRAP />



RTC STRIKES BACK

Objectives

By the end of this 3-hour session, you will be able to:

- ✓ Understand what CI/CD is and why it matters;
- ✓ Set up a GitHub Actions pipeline for your project;
- ✓ Implement automated testing and code quality checks;
- ✓ Create automated releases with binaries;
- ✓ Apply these concepts to your backend.

⚠️

This bootstrap focuses on the **backend**. You will need to apply similar concepts to your frontend/desktop application on your own.

Part 1: Research & Discovery

Research Exercise

Instruction: Work in your team to research and answer these questions. You can use:

- ✓ Google,
- ✓ GitHub documentation,
- ✓ Any resources you find useful.

> No answer will be provided during this time. You must find the information yourselves.

Block 1: Understanding CI/CD

- ✓ **Q1.1**: What does ”CI/CD” stand for? What is the difference between CI and CD?
- ✓ **Q1.2**: Name 3 real benefits of using CI/CD in a software project. Be specific - how does it help developers day-to-day
- ✓ **Q1.3**: What happens if you DON'T use CI/CD? What problems might you encounter?

Block 2: GitHub Actions Basics

- ✓ **Q2.1**: What are GitHub Actions? How is it different from just using Git/GitHub?
- ✓ **Q2.2**: What file format is used to define GitHub Actions workflows? Where should these files be in your repository?
- ✓ **Q2.3**: What are the 3 main components of a GitHub Actions workflow? (Hint: start with W, J and S)
- ✓ **Q2.4**: What is a ”runner”? What operating systems can runners use?

Block 3: Workflow Triggers

- ✓ **Q3.1**: What is a workflow ”trigger”? Give 3 examples of events that can trigger a workflow.
- ✓ **Q3.2**: How would you configure a workflow to run ONLY when code is pushed to the `main` branch?
- ✓ **Q3.3**: How would you configure a workflow to run when someone creates a pull request?
- ✓ **Q3.4**: What is a ”tag” in Git? How could you use tags to trigger a release workflow?

Part 2: Your First Pipeline

Exercise 1: Create a Basic CI Workflow

Now that you understand the concepts, let's build your first workflow!

- ✓ **Step 1**: Create the directory structure;
- ✓ **Step 2**: Create `.github/workflows/ci.yml`;
- ✓ **Step 3**: Commit and Push;
- ✓ **Step 4**: Watch it run!

Exercise 2: Add a Status Badge

- ✓ **Step 1**: Get the badge;
- ✓ **Step 2**: Add to README.md;
- ✓ **Step 3**: Commit and Push.

You should now see your badge on your `README`

- ✓ Green checkmark if CI passes;
- ✓ Red X if CI fails;
- ✓ Yellow dot if running.

> Try to find how to run multiple jobs in parallel and in sequential. Think about why you should use multiple jobs.

Part 3: Testing & Quality Checks

Exercise 3: Add Code Quality Checks

Enhanced CI with Quality Checks. For this exercise you will have to do 3 jobs:

Quality Checks

- ✓ Run format checking.
- ✓ Run lint checking.
- ✓ Type checking.

Build and Test

- ✓ Cache your dependencies.
- ✓ Build your app.
- ✓ Run tests.

Security Audit

- ✓ Run a security audit.

⚠️

NodeJS and Rust can do all those actions but don't run the same way
Some modules may not be installed by default on the self-hosted machine

Exercise 4: Test a Failing Pipeline

Let's intentionally break the code to see how CI catches it!

Step 1: Add poorly formatted code

```rust
fn test_function( )   {
let x=1+2;
    let y    =    3      +      4;
println!("Hello");
}
```

```javascript
function testFunction(  )   {
const x=1+2
    let y    =    3      +      4;
console.log("Hello");
}
```

Step 2: Commit and Push

```bash
git add FILE_NAME && git commit -m "DESCRIPTIVE_MESSAGE" && git push
```

Step 3: Watch it fail

✓ See the Red Cross on the *Actions* tab, then find exactly which check failed in the workflow.

Step 4: Fix it

```bash
cargo fmt
cargo clippy
cargo clippy --fix
git add .
git commit -m "Fix formatting and linting issues"
git push
```

```bash
npm run format
npm run lint:fix
git add .
git commit -m "Fix formatting and linting issues"
git push
```

Step 5: Watch it pass Actions tab -> See the Green Checkmark

Part 4: Automated Releases

Exercise 5: Create a Release Workflow

Create `.github/workflows/release.yml`

For this exercise, you will have to create 2 jobs:

Create Release

- ✓ Generate the Changelog.
- ✓ Create the release.

Build Binaries

- ✓ Build the package.
- ✓ Upload the release.

Test it by creating a release on git

```
# Make sure your code is committed and pushed
git add .
git commit -m "Prepare for v1.0.0 release"
git push

# Create and Push a tag
git tag v1.0.0
git push origin v1.0.0
```

Part 5: Hands-on Practice

Exercise 6: Complete Backend CI/CD

Your Task: Set up a complete CI/CD for your application backend

Testing Steps:

```
# 1. Add the workflows
git add .github/workflows/
git commit -m "Add CI/CD pipelines"
git push

# 2. Verify CI passes
# -> Check Actions tab

# 3. Create first release
git tag add v0.1.0
git push origin v0.1.0

# 4. Verify release is created
# -> Check Releases tab
```

Success Criteria:

- ✓ Green badge in README.
- ✓ CI runs on every push.
- ✓ First release created successfully.

v 1.0.1

{EPITECH}